

InsightFlow Agent 输出产物文档

1. 文档目标

本文档用于说明 **InsightFlow Agent: Multi-Agent Tool-Calling BI Workflow** 在不同阶段会生成哪些输出产物。

InsightFlow Agent 的核心不是生成一段聊天回复，而是生成一组可追踪、可验证、可评估的 **Agent Workflow Artifacts**。

这些产物包括：

```
generated_sql  
review_result  
execution_result  
fixed_sql  
final_answer  
trace.json  
eval/report.md  
chart.png  
report.md  
weekly_business_report.md  
action_plan.json  
approval record  
tasks record  
metric_alerts record  
audit_logs record
```

这些产物共同服务于项目核心目标：

```
Multi-Agent  
Tool Calling  
SQL Execution  
Execution Feedback Repair  
Trace  
Eval  
Report Generation  
Action Workflow
```

2. 输出产物总览

InsightFlow 的输出产物分为两类：

2.1 每次 Agent Run 生成的运行产物

用户每输入一个业务问题，系统会创建一次 `run`。

一次 run 可能生成：

```
generated_sql  
review_result  
execution_result  
fixed_sql  
final_answer  
trace.json  
chart.png  
report.md  
action_plan.json  
approval record  
task / alert record  
audit log
```

2.2 项目仓库交付产物

项目作为简历项目，需要在仓库中交付：

```
可运行代码  
SQLite 示例数据库  
Agent / Tool 模块  
LangGraph workflow  
Streamlit Demo  
Eval 测试集  
Eval 报告  
Trace 样例  
README  
需求文档  
开发计划文档  
应用场景文档  
参考项目文档  
输出产物文档
```

3. P0 输出产物：Agentic SQL Core

P0 阶段目标：

```
证明 InsightFlow 不是普通 Text2SQL，  
而是 Multi-Agent + Tool Calling + SQL Execution Feedback Workflow。
```

P0 每次 run 主要生成：

```
generated_sql  
review_result  
execution_result  
fixed_sql, 可选  
final_answer  
trace.json  
eval/report.md
```

Part 1: P0 核心输出产物

4. generated_sql

4.1 说明

`generated_sql` 是 SQL Generator Agent 根据用户问题、数据库 Schema 和业务指标定义生成的 SQL。

它不是最终答案，而是后续 `validate_sql()` 和 `run_sql()` 的输入。

4.2 示例用户问题

最近 30 天销售额最高的 5 个商品是什么？

4.3 示例 generated_sql

```
SELECT  
  p.product_name,  
  SUM(oi.quantity * oi.unit_price) AS sales  
FROM orders o  
JOIN order_items oi ON o.id = oi.order_id  
JOIN products p ON oi.product_id = p.id  
WHERE o.status = 'paid'  
  AND o.order_date >= date('now', '-30 days')  
GROUP BY p.product_name  
ORDER BY sales DESC  
LIMIT 5;
```

4.4 保存位置

```
State.generated_sql  
logs/traces/{run_id}.json  
Streamlit SQL 展示区
```

4.5 体现的能力

```
SQL Generator Agent  
Metric-aware SQL generation  
业务指标口径使用  
多表 JOIN  
结构化输出
```

5. review_result

5.1 说明

`review_result` 是 SQL Reviewer Agent 调用 `validate_sql()` 工具后的校验结果。

它用于证明系统不会直接执行 LLM 生成的 SQL，而是先经过安全校验和业务口径校验。

5.2 成功示例

```
{  
  "approved": true,  
  "risk_level": "low",  
  "issues": [],  
  "checks": {  
    "select_only": true,  
    "no_dangerous_keywords": true,  
    "single_statement": true,  
    "tables_exist": true,  
    "columns_exist": true,  
    "has_limit": true,  
    "metric_formula_correct": true,  
    "paid_filter_included": true,  
    "sensitive_fields_blocked": true  
  },  
  "normalized_sql": "SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS sales ..."  
}
```

5.3 拒绝示例

```
{
  "approved": false,
  "risk_level": "high",
  "issues": [
    "SQL contains DELETE statement",
    "Only SELECT queries are allowed"
  ],
  "suggestion": "Regenerate a read-only SELECT query."
}
```

5.4 保存位置

```
State.review_result
logs/traces/{run_id}.json
Streamlit SQL Review 展示区
eval/report.md
```

5.5 体现的能力

```
SQL Reviewer Agent
validate_sql() Tool Calling
SQL 安全边界
业务指标口径校验
敏感字段拦截
执行前防护
```

6. execution_result

6.1 说明

`execution_result` 是 SQL Executor 调用 `run_sql()` 工具后返回的真实数据库执行结果。

它是最终回答的唯一数据来源。

6.2 成功示例

```
{
  "success": true,
  "columns": ["product_name", "sales"],
  "rows": [
    ["Camera A", 128000],
    ["Lens B", 113500],
    ["Headphone C", 98000],
  ]
}
```

```
["Tripod D", 87500],  
["Keyboard E", 79600]  
],  
"row_count": 5,  
"execution_time_ms": 34  
}
```

6.3 失败示例

```
{  
  "success": false,  
  "error": "no such column: oi.price",  
  "execution_time_ms": 11  
}
```

6.4 保存位置

```
State.execution_result  
State.error_message  
logs/traces/{run_id}.json  
Streamlit SQL 执行结果区  
eval/report.md
```

6.5 体现的能力

```
run_sql() Tool Calling  
真实数据库执行  
工具返回 Observation  
执行耗时记录  
错误反馈
```

7. fixed_sql

7.1 说明

`fixed_sql` 是 SQL 执行失败后，由 Error Fix Agent 根据数据库错误、原始 SQL、Schema 和用户问题生成的修复 SQL。

这是 P0 最重要的技术亮点之一。

7.2 错误 SQL 示例

```
SELECT  
  p.product_name,
```

```
SUM(oi.quantity * oi.price) AS sales
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.status = 'paid'
GROUP BY p.product_name
ORDER BY sales DESC
LIMIT 5;
```

数据库错误：

```
no such column: oi.price
```

7.3 fixed_sql 示例

```
{
  "fixed_sql": "SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS sales FROM orders o
JOIN order_items oi ON o.id = oi.order_id JOIN products p ON oi.product_id = p.id WHERE o.status =
'paid' GROUP BY p.product_name ORDER BY sales DESC LIMIT 5;",
  "fix_reason": "order_items 表中不存在 price 字段，正确字段是 unit_price。",
  "retry_count": 1
}
```

7.4 修复链路

```
SQL Generator Agent
-> SQL Reviewer Agent
-> SQL Executor Tool
-> run_sql() failed
-> Error Fix Agent
-> SQL Reviewer Agent
-> SQL Executor Tool
-> Insight Agent
```

7.5 保存位置

```
State.fixed_sql
State.retry_count
logs/traces/{run_id}.json
Streamlit 错误修复过程区
eval/report.md
```

7.6 体现的能力

Action-Observation-Revision
执行反馈修复
Error Fix Agent
LangGraph Conditional Edge
最多重试一次
非普通 Text2SQL

8. final_answer

8.1 说明

`final_answer` 是 Insight Agent 基于 `execution_result` 生成的自然语言回答。

它必须严格基于 SQL 查询结果，不允许编造。

8.2 成功示例

最近 30 天销售额最高的 5 个商品分别是 Camera A、Lens B、Headphone C、Tripod D 和 Keyboard E。
其中 Camera A 销售额最高，为 128,000 元。

8.3 失败示例

本次查询未能成功完成。系统在执行 SQL 时遇到字段不存在错误，并尝试根据 Schema 自动修复，但修复后仍未执行成功。失败原因：no such column: xxx。

8.4 保存位置

State.final_answer
logs/traces/{run_id}.json
Streamlit 最终回答区

8.5 体现的能力

Insight Agent
Grounded Answer
基于工具结果回答
失败时不编造

9. trace.json

9.1 说明

`trace.json` 是每次 Agent run 的完整执行轨迹。

它是项目最重要的工程产物之一，因为它能展示：

- 每个 Agent 节点
- 每次工具调用
- 工具输入输出摘要
- 错误
- 重试
- 耗时
- 最终状态

9.2 文件路径

`logs/traces/{run_id}.json`

示例：

`logs/traces/run_20260618_001.json`

9.3 Trace 示例

```
{
  "run_id": "run_001",
  "session_id": "session_001",
  "user_question": "最近 30 天销售额最高的 5 个商品是什么？",
  "status": "success",
  "nodes": [
    {
      "node": "schema_agent",
      "status": "success",
      "latency_ms": 12
    },
    {
      "node": "metric_agent",
      "status": "success",
      "latency_ms": 8
    },
    {
      "node": "sql_generator",
      "status": "success",
      "latency_ms": 1800
    }
  ]
}
```

```

},
{
  "node": "sql_reviewer",
  "status": "success",
  "latency_ms": 20
},
{
  "node": "sql_executor",
  "status": "success",
  "latency_ms": 34
},
{
  "node": "insight_agent",
  "status": "success",
  "latency_ms": 1200
}
],
"tool_calls": [
  {
    "tool_name": "get_database_schema",
    "input_summary": "db_path=data/ecommerce.db",
    "output_summary": "5 tables loaded",
    "status": "success",
    "latency_ms": 12
  },
  {
    "tool_name": "retrieve_metric_definition",
    "input_summary": "metric=sales",
    "output_summary": "GMV definition loaded",
    "status": "success",
    "latency_ms": 8
  },
  {
    "tool_name": "validate_sql",
    "input_summary": "SELECT ...",
    "output_summary": "approved=true",
    "status": "success",
    "latency_ms": 20
  },
  {
    "tool_name": "run_sql",
    "input_summary": "SELECT ...",
    "output_summary": "5 rows returned",
    "status": "success",
    "latency_ms": 34
  }
],
"retry_count": 0,
"total_latency_ms": 3074,
"final_status": "success"
}

```

9.4 失败修复 Trace 示例

```
schema_agent succeeded
metric_agent succeeded
sql_generator succeeded
sql_reviewer approved
sql_executor failed: no such column: oi.price
error_fixer generated fixed_sql
sql_reviewer approved fixed_sql
sql_executor succeeded
insight_agent succeeded
```

9.5 体现的能力

Agent workflow 可观测
Tool Calling 可追踪
错误修复路径可复盘
Eval 数据来源
面试展示材料

10. eval/report.md

10.1 说明

`eval/report.md` 是 AgentEval 脚本对测试集运行后的评测报告。

它用于证明项目不是只靠手工 Demo，而是有可重复评测。

10.2 文件路径

```
eval/report.md
```

10.3 P0 Eval 指标

```
total_cases
success_count
success_rate
sql_execution_success_rate
sql_first_pass_success_rate
sql_repair_success_rate
dangerous_sql_block_rate
metric_definition_accuracy
average_tool_calls
```

```
average_latency_ms
failure_type_distribution
```

10.4 示例报告

InsightFlow Agent Eval Report

Summary

- Total cases: 20
- Success cases: 16
- Success rate: 80%
- SQL execution success rate: 85%
- SQL first-pass success rate: 70%
- SQL repair success rate: 75%
- Dangerous SQL block rate: 100%
- Metric definition accuracy: 90%
- Average tool calls per run: 4.8
- Average latency: 3.2s

Failure Types

Failure Type	Count
column_not_found	2
metric_mismatch	1
empty_result	1

Key Observations

- SQL repair works well for simple column mismatch errors.
- Dangerous SQL was blocked in all safety cases.
- Metric-aware SQL generation reduced GMV calculation errors.

10.5 体现的能力

项目可评测
SQL 成功率
修复成功率
工具调用稳定性
安全拦截能力
工程化意识

Part 2: P1 输出产物

P1 阶段目标:

证明系统不只是会查 SQL，
还能生成可靠、可追溯的业务分析报告。

P1 会额外生成：

```
business_context.json
evidence_result.json
chart.png
report.md
```

11. business_context.json

11.1 说明

`business_context.json` 是 Context Retrieval Agent 调用 `retrieve_business_context()` 后得到的业务上下文。

MVP 可以由以下文件组成：

```
data/metrics.yaml
data/business_rules.md
data/table_docs.md
data/sql_examples.json
```

11.2 示例

```
{
  "metric_definitions": [
    {
      "metric": "GMV",
      "formula": "SUM(order_items.quantity * order_items.unit_price)",
      "filters": ["orders.status = 'paid'"],
      "exclude": ["cancelled", "refunded"]
    }
  ],
  "business_rules": [
    "销售额默认只统计 paid 订单",
    "趋势分析默认按月聚合",
    "下降分析需要比较当前周期和上一周期"
  ],
  "sql_examples": [
    {
      "question": "最近 30 天销售额最高的商品",
      "sql_pattern": "orders JOIN order_items JOIN products GROUP BY product_name"
    }
  ]
}
```

```
]
}
```

11.3 保存位置

```
State.business_context
logs/traces/{run_id}.json
reports/markdown/{run_id}_report.md
```

11.4 体现的能力

Business Context Retrieval
Metric-aware SQL
业务口径可控
不是只靠 Schema 生成 SQL

12. evidence_result.json

12.1 说明

`evidence_result.json` 是 Evidence Validator Agent 的输出。

它用于区分：

数据支持结论
需要进一步验证的假设
被阻止的 unsupported claims

12.2 示例

```
{
  "data_supported_findings": [
    {
      "claim": "Camera 品类 GMV 下降 32.8%",
      "evidence": "SQL result: category=Camera, gmv_drop_rate=0.328",
      "confidence": 0.95
    },
    {
      "claim": "下降主要来自订单量下降",
      "evidence": "order_count_drop_rate=0.29, aov_drop_rate=0.052",
      "confidence": 0.88
    }
  ],
  "hypotheses": [
```

```
{
  "claim": "可能与广告流量下降有关",
  "reason": "当前数据库没有曝光、点击率和转化率数据，无法验证",
  "needs_more_data": ["ad_impressions", "ctr", "conversion_rate"]
},
{
  "claim": "可能与库存不足有关",
  "reason": "当前数据库没有库存数据，无法验证",
  "needs_more_data": ["inventory_level"]
}
],
"unsupported_claims_blocked": [
  "库存不足是导致销量下降的主要原因"
],
"actionable": true
}
```

12.3 保存位置

```
State.evidence_result
logs/traces/{run_id}.json
reports/markdown/{run_id}_report.md
```

12.4 体现的能力

```
Evidence Validator Agent
防止业务洞察幻觉
数据支持结论与假设分离
unsupported_claim_rate 可评测
```

13. chart.png

13.1 说明

`chart.png` 是 Chart Agent 调用 `generate_chart()` 后生成的图表文件。

13.2 文件路径

```
reports/charts/{run_id}_{chart_name}.png
```

示例：

```
reports/charts/run_001_category_gmv_trend.png
```

13.3 支持图表类型

P1 支持:

```
bar  
line  
pie, 可选
```

推荐规则:

```
排名问题 -> bar  
趋势问题 -> line  
占比问题 -> pie  
下降分析 -> line + bar
```

13.4 保存位置

```
reports/charts/  
State.chart_paths  
logs/traces/{run_id}.json  
reports/markdown/{run_id}_report.md
```

13.5 体现的能力

```
Chart Agent  
generate_chart() Tool Calling  
SQL 结果可视化  
报告交付物
```

14. report.md

14.1 说明

`report.md` 是 Report Agent 调用 `save_report()` 后保存的 Markdown 分析报告。

P1 的报告不是完整经营周报，而是针对一次分析任务生成的可追溯分析报告。

14.2 文件路径

```
reports/markdown/{run_id}_report.md
```

示例:

reports/markdown/run_001_decline_analysis.md

14.3 报告结构

销售下降分析报告

1. 用户问题

最近 3 个月销售额下降最多的品类有哪些？

2. 使用的业务指标

- GMV = SUM(order_items.quantity * order_items.unit_price)
- 仅统计 orders.status = 'paid' 的订单

3. 执行 SQL

```
```sql
```

```
SELECT ...
```

## 4. 查询结果摘要

Camera 品类 GMV 下降 32.8%。

## 5. 数据支持结论

- Camera 品类 GMV 下降 32.8%
- 下降主要来自订单量下降

## 6. 需要进一步验证的假设

- 可能与广告流量下降有关，但当前数据无法验证
- 可能与库存不足有关，但当前数据无法验证

## 7. 图表

reports/charts/run\_001\_category\_gmv\_trend.png

## 8. 下一步建议

- 排查 Camera 品类流量来源变化
- 检查重点商品库存和价格变化

## 9. Trace

logs/traces/run\_001.json

### ### 14.4 保存位置

```
```text
reports/markdown/
State.report_path
logs/traces/{run_id}.json
```

14.5 体现的能力

Report Agent
save_report() Tool Calling
可交付业务分析产物
结论可追溯到 SQL 和数据
不是 LLM 直接编报告

Part 3: P2 输出产物

P2 阶段目标:

证明系统可以从分析走向业务报告和业务行动，
但仍然保持安全边界和审批机制。

P2 会额外生成:

```
weekly_business_report.md
action_plan.json
approval record
tasks record
metric_alerts record
action_verification_result.json
audit_logs record
```

15. weekly_business_report.md

15.1 说明

`weekly_business_report.md` 是 Report Supervisor Agent 拆解多个分析子任务后，由 Report Agent 保存的经营周报。

它不是 P0 功能，而是 P2 的报告型升级能力。

15.2 示例用户输入

帮我生成一份本周电商经营分析周报，包括销售额、订单量、Top 商品、下降品类、渠道表现和运营建议。

15.3 输出路径

reports/markdown/weekly_business_report_{week}.md

示例：

reports/markdown/weekly_business_report_2026_w25.md

15.4 周报结构

本周电商经营分析周报

1. 核心结论

2. 核心指标

指标	本周	上周	环比
GMV	328,000	302,600	+8.4%
订单量	1,240	1,180	+5.1%
客单价	264.5	256.4	+3.1%

3. Top 商品

4. Top 品类

5. 销售下降品类

6. 渠道表现

7. 数据支持结论

8. 需要进一步验证的假设

9. 下周建议

10. SQL 与 Trace 记录

15.5 相关产物

```
reports/charts/weekly_sales_trend.png
reports/charts/top_products.png
reports/charts/category_decline.png
reports/data/weekly_metrics.csv
logs/traces/{run_id}.json
```

15.6 体现的能力

Report Supervisor Agent
多 SQL 子任务编排
多工具调用
经营分析交付物
可追溯周报

16. action_plan.json

16.1 说明

`action_plan.json` 是 Action Planner Agent 根据 Evidence Validator 结果生成的结构化行动计划。

16.2 示例

```
{
  "run_id": "run_001",
  "action_plan": [
    {
      "action_type": "create_task",
      "title": "排查 Camera 品类订单量下降原因",
      "reason": "Camera 品类 GMV 下降 32.8%，订单量下降 29%",
      "evidence_ids": ["finding_001", "finding_002"],
      "priority": "high",
      "owner": "ops_team",
      "due_date": "2026-06-25"
    },
    {
      "action_type": "create_metric_alert",
      "metric": "category_gmv_drop_rate",
      "target": "Camera",
      "condition": "drop_rate > 20%",
      "window": "7d",
      "severity": "medium"
    }
  ]
}
```

```
]
}
```

16.3 保存位置

```
State.action_plan
reports/actions/{run_id}_action_plan.json
logs/traces/{run_id}.json
```

16.4 体现的能力

Action Planner Agent
分析结果到业务动作
结构化行动计划
证据关联

17. approval record

17.1 说明

`approval record` 用于保存高风险动作前的人工确认结果。

P2 后，以下动作需要审批：

```
create_task
create_metric_alert
create_email_draft
export_large_result
schedule_followup_run
```

17.2 示例

```
{
  "approval_id": "approval_001",
  "run_id": "run_001",
  "pending_actions": ["create_task", "create_metric_alert"],
  "summary": "为 Camera 品类创建 1 个运营排查任务和 1 个 GMV 下降监控告警。",
  "risk_level": "medium",
  "decision": "approve",
  "comment": "同意创建任务和告警",
  "created_at": "2026-06-18T12:00:00"
}
```

17.3 保存位置

```
SQLite approvals table
logs/traces/{run_id}.json
audit_logs
```

17.4 体现的能力

```
Human-in-the-loop
Approval Gate
风险控制
状态变更前审批
```

18. tasks record

18.1 说明

`tasks record` 是 `create_task()` 工具执行后的结果。

第一版可以写入 SQLite 的 `tasks` 表。

18.2 示例

```
{
  "task_id": "task_001",
  "run_id": "run_001",
  "title": "排查 Camera 品类订单量下降原因",
  "description": "Camera 品类最近 3 个月 GMV 下降 32.8%，订单量下降 29%，AOV 仅下降 5.2%，初步判断主要问题来自订单量下降。",
  "priority": "high",
  "owner": "ops_team",
  "status": "created",
  "report_path": "reports/markdown/run_001_decline_analysis.md",
  "created_at": "2026-06-18T12:01:00"
}
```

18.3 保存位置

```
SQLite tasks table
State.created_actions
logs/traces/{run_id}.json
audit_logs
```

18.4 体现的能力

create_task() Tool Calling
真实状态变更
业务行动对象
行动型 Agent

19. metric_alerts record

19.1 说明

metric_alerts record 是 create_metric_alert() 工具执行后的结果。

第一版可以写入 SQLite 的 metric_alerts 表。

19.2 示例

```
{
  "alert_id": "alert_001",
  "run_id": "run_001",
  "metric": "category_gmv_drop_rate",
  "target": "Camera",
  "condition": "drop_rate > 20%",
  "window": "7d",
  "severity": "medium",
  "status": "active",
  "created_at": "2026-06-18T12:02:00"
}
```

19.3 保存位置

SQLite metric_alerts table
State.created_actions
logs/traces/{run_id}.json
audit_logs

19.4 体现的能力

create_metric_alert() Tool Calling
一次性分析转持续监控
业务 workflow action

20. action_verification_result.json

20.1 说明

`action_verification_result.json` 是 Action Verifier Agent 对行动执行结果的验证。

它用于确认任务和告警是否真的创建成功。

20.2 示例

```
{
  "run_id": "run_001",
  "action_verified": true,
  "created_objects": [
    {
      "type": "task",
      "id": "task_001",
      "status": "created"
    },
    {
      "type": "metric_alert",
      "id": "alert_001",
      "status": "active"
    }
  ],
  "verification_checks": {
    "task_created": true,
    "alert_created": true,
    "evidence_attached": true,
    "report_linked": true,
    "approval_recorded": true
  }
}
```

20.3 保存位置

```
reports/actions/{run_id}_action_verification_result.json
State.created_actions
logs/traces/{run_id}.json
audit_logs
```

20.4 体现的能力

Action Verifier Agent
行动是否成功可验证
不是只生成建议
对标 run_tests / verification 思路

21. audit_logs record

21.1 说明

`audit_logs record` 记录审批和行动工具调用的全过程。

21.2 示例

```
{
  "audit_log_id": "audit_001",
  "run_id": "run_001",
  "event_type": "action_executed",
  "action": "create_task",
  "approval_id": "approval_001",
  "created_object_id": "task_001",
  "status": "success",
  "timestamp": "2026-06-18T12:03:00"
}
```

21.3 保存位置

SQLite audit_logs table
logs/traces/{run_id}.json

21.4 体现的能力

Audit Log
行动可追踪
审批可复盘
企业级安全边界

Part 4: P3 输出产物

P3 阶段目标：

证明项目具备工具标准化、异步执行、API 化和工程化能力。

P3 会额外生成：

MCP server code
FastAPI API
run status records

```
async job status
trace dashboard
Docker deployment files
CI test results
```

22. MCP server code

22.1 说明

P3 会把部分外部工具封装成 MCP-compatible tools。

22.2 MCP Server 规划

```
database-mcp-server:
  get_database_schema
  get_sample_rows
  run_sql

report-mcp-server:
  generate_chart
  save_report

action-mcp-server:
  create_task
  create_metric_alert
  create_email_draft
```

22.3 不优先 MCP 化的模块

```
validate_sql
permission_checker
log_trace
eval_runner
```

原因：

这些更适合作为系统内部安全、审计和评测模块。

23. API / async run records

23.1 说明

P3 增加 FastAPI 和异步任务后，每次 run 会产生运行状态记录。

23.2 状态

```
queued
running
waiting_for_approval
completed
failed
cancelled
```

23.3 API

```
POST /api/runs
GET /api/runs/{run_id}
GET /api/runs/{run_id}/trace
GET /api/runs/{run_id}/events
POST /api/runs/{run_id}/cancel
```

24. Trace Dashboard 产物

24.1 说明

Trace Dashboard 用于展示系统运行情况。

24.2 展示内容

```
Agent 节点耗时
工具调用次数
SQL 执行耗时
SQL 修复次数
失败类型分布
Eval 成功率
Action 审批记录
Audit Log
```

Part 5：项目仓库交付物

25. P0 仓库交付物

P0 阶段仓库应包含：

```

insightflow-agent/
├── README.md
├── REQUIREMENTS.md
├── DEVELOPMENT_PLAN.md
├── USE_CASES.md
├── OUTPUTS_AND_ARTIFACTS.md
├── REFERENCE_PROJECTS.md
├── requirements.txt
├── .env.example
├── app.py
├──
├── data/
│   ├── ecommerce.db
│   ├── seed_data.py
│   └── metrics.yaml
├──
├── agents/
│   ├── supervisor.py
│   ├── schema_agent.py
│   ├── metric_agent.py
│   ├── sql_generator.py
│   ├── sql_reviewer.py
│   ├── error_fixer.py
│   └── insight_agent.py
├──
├── tools/
│   ├── schema_tool.py
│   ├── metric_tool.py
│   ├── sql_validator.py
│   ├── sql_executor.py
│   └── trace_logger.py
├──
├── graph/
│   ├── state.py
│   ├── nodes.py
│   └── workflow.py
├──
├── eval/
│   ├── test_questions.json
│   ├── run_eval.py
│   └── report.md
├──
├── logs/
│   └── traces/
├──
├── reports/
│   ├── charts/
│   └── markdown/
├──
└── tests/

```

```
|— test_schema_tool.py
|— test_metric_tool.py
|— test_sql_validator.py
|— test_sql_executor.py
|— test_workflow.py
└— test_eval.py
```

26. P1 扩展交付物

```
agents/
|— context_retriever.py
|— evidence_validator.py
|— chart_agent.py
└— report_agent.py

tools/
|— context_tool.py
|— python_analysis_tool.py
|— chart_tool.py
└— report_tool.py

data/
|— business_rules.md
|— table_docs.md
└— sql_examples.json

reports/
|— charts/
└— markdown/
```

27. P2 扩展交付物

```
agents/
|— report_supervisor.py
|— action_planner.py
|— risk_assessor.py
└— action_verifier.py

tools/
|— action_tool.py
|— approval_tool.py
└— audit_logger.py

SQLite tables:
|— reports
```

```
|— tasks
|— metric_alerts
|— approvals
|— audit_logs
|— scheduled_runs
```

28. P3 扩展交付物

```
mcp_servers/
|— database_server.py
|— report_server.py
|— action_server.py

api/
|— main.py

dashboard/
Dockerfile
docker-compose.yml
.github/workflows/
```

Part 6: 阶段输出总结

29. P0 输出总结

P0 重点证明:

```
Multi-Agent + Tool Calling + SQL Execution Feedback
```

必须生成:

```
generated_sql
review_result
execution_result
fixed_sql, 可选
final_answer
trace.json
eval/report.md
```

30. P1 输出总结

P1 重点证明:

Reliable Analysis + Report Generation

额外生成:

business_context.json
evidence_result.json
chart.png
report.md

31. P2 输出总结

P2 重点证明:

Business Review + Action Workflow

额外生成:

weekly_business_report.md
action_plan.json
approval record
tasks record
metric_alerts record
action_verification_result.json
audit_logs record

32. P3 输出总结

P3 重点证明:

MCP + Engineering

额外生成:

MCP server code
FastAPI API
run status records

async job status
trace dashboard
Docker deployment files
CI test results

Part 7: 面试表达

33. 一句话说明项目会生成什么

InsightFlow 每次运行不会只生成一段回答，而是会生成 SQL、SQL Review 结果、真实数据库执行结果、错误修复记录、最终分析结论、Trace 日志和 Eval 报告。升级后还会生成图表、Markdown 分析报告、Evidence 校验结果、经营周报、行动计划、审批记录、任务、指标告警和审计日志。

34. 技术表达

我把每次 Agent 执行都设计成一个完整 workflow artifact。Agent 生成 SQL 后不会直接返回，而是经过 validate_sql 工具校验，再由 run_sql 工具真实执行。工具返回的 execution_result 或 error 会写入 LangGraph State，并通过条件边决定进入 Insight Agent 还是 Error Fix Agent。整个过程会生成 trace.json 和 eval report，用于复盘 tool calls、节点耗时、失败类型和修复成功率。

35. 强调 Multi-Agent + Tool Calling 的表达

这个项目的生成物不是普通答案，而是多 Agent 工具调用链路的结果。Schema Agent 生成 schema context，Metric Agent 生成指标上下文，SQL Generator 生成 SQL，SQL Reviewer 生成 review_result，SQL Executor 生成 execution_result，Error Fix Agent 生成 fixed_sql，Insight Agent 生成 final_answer。每个工具调用都会被记录到 trace，并进入 eval 统计。

36. 强调报告生成的表达

P1 阶段系统会生成可追溯 Markdown 分析报告。报告不是由 LLM 直接编写，而是由 Report Agent 汇总 SQL、execution_result、图表路径、数据支持结论、假设和 Trace 链接生成。核心结论都可以回溯到 SQL 执行结果。

37. 强调行动型升级的表达

P2 阶段系统不仅会生成分析报告，还会根据 Evidence Validator 的结果生成 action_plan。经过 Risk Assessor 和 Approval Gate 后，Action Executor 会创建任务和指标告警，Action Verifier 会验证这些对象是否真的创建成功，Audit Logger 会记录审批和执行过程。

38. 最终结论

InsightFlow 的核心输出不是：

一个自然语言回答

而是：

一组可追踪、可验证、可评估的 Agent Workflow Artifacts。

P0 阶段最核心的输出物是：

```
generated_sql  
review_result  
execution_result  
fixed_sql  
trace.json  
eval/report.md
```

P1 阶段增加：

```
chart.png  
report.md  
evidence_result.json
```

P2 阶段增加：

```
weekly_business_report.md  
action_plan.json  
tasks  
metric_alerts  
approval records  
audit logs
```

这些产物共同服务于项目第一目标：

突出 Multi-Agent 和 Tool Calling。